

人工智能：

强人工智能：综合多方面的人工智能 目前较弱

弱人工智能：某个方面的人工智能 目前较强

图灵机定义：

- 是一个数学概念
- 一个七元组：纸带、符号、读写头、规则、状态、起始、结束。（存储、符号、读写、程序、数据、开始、结束）

人工智能三代流派：

- **符号主义**
 - 把现实的物，映射到代表它的符号，在符号上完成所有的推理、计算等等
 - 图灵测试就是这个思路
- **连接主义**
 - 人脑就是神经元和之间的一些连接
 - 模拟人脑的做法，就可以模拟出人脑的功能
 - 于是提出了神经网络模型
- **行为主义**
 - 智能来自更低级的感知和行动，表现得好就行

大数据

指无法在一定时间范围内用常规软件工具进行捕捉、管理和处理的数据集合，是需要新处理模式才能具有更强的决策力、洞察发现力和流程优化能力的海量、高增长率和多样化的信息资产

大数据的5V特点（IBM提出）：Volume（大量）、Velocity（高速）、Variety（多样）、Value（低价值密度）、Veracity（真实性）

摩尔定律：

当价格不变时，集成电路上可容纳的元器件的数目，约每隔18-24个月便会增加一倍，性能也将提升一倍。

计算机分类：

- 1、巨型化（1946-1980的重点）
- 2、微型化（1980-2000的重点）
- 3、网络化（1990-现在 重点）
- 4、人工智能化（2015-现在 重点）
- 5、多样化

专家系统的基本组成

组成三要素：

- 一个综合数据库——存放信息
- 一组产生式规则——知识
- 一个控制系统——规则的解释或执行程序（控制策略）（推理引擎）

基本过程：

产生式系统的基本过程

过程PRODUCTION

- 1, DATA←初始数据库
- 2, until DATA满足结束条件, do
- 3, {
- 4, 在规则集中选择一条可应用于DATA 的规则R
- 5, DATA ←R应用到DATA得到的结果
- 6, }

产生式系统的特点

- 数据驱动
- 知识的无序性
- 控制系统与问题无关
- 数据、知识和控制相互独立

产生式系统的类型

- 正向、逆向、双向产生式系统
- 可交换的产生式系统
- 可分解的产生式系统

回溯策略

皇后问题

图搜索策略

保留已经搜索过的路径

无信息图搜索过程

宽度优先搜索

- 当问题有解时，一定能找到解
- 当问题为单位耗散值，且问题有解时，一定能找到最优解
- 方法与问题无关，具有通用性
- 效率较低
- 属于图搜索方法

深度优先搜索

- 一般不能保证找到最优解
- 当深度限制不合理时，可能找不到解，可以将算法改为可变深度限制
- 最坏情况时，搜索空间等同于穷举
- 与回溯法的差别：图搜索
- 是一个通用的与问题无关的方法

渐进式深度优先搜索方法

- 目的
- 解决宽度优先方法的空间问题和回溯方法不能找到最优解的问题。
- 思想
首先给回溯法一个比较小的深度限制，然后逐渐增加深度限制，直到找到解或找遍所有分支为止。

启发式图搜索：

- 利用知识来引导搜索，达到减少搜索范围，降低问题复杂度的目的。
- 启发信息的强度
- 强：降低搜索工作量，但可能导致找不到最优解
- 弱：一般导致工作量加大，极端情况下变为盲目搜索，但可能可以找到最优解

A算法

$$f(n) = g(n) + h(n)$$

- $g^*(n)$: 从s到n的最短路径的耗散值
- $h^*(n)$: 从n到g的最短路径的耗散值
- $f^*(n)=g^*(n)+h^*(n)$: 从s经过n到g的最短路径的耗散值
- $g(n)$ 、 $h(n)$ 、 $f(n)$ 分别是 $g^*(n)$ 、 $h^*(n)$ 、 $f^*(n)$ 的估计值

最佳图搜索算法A* (A**算法)

- 在A算法中, 如果满足条件:

$$h(n) \leq h^*(n)$$

则A算法称为A*算法

$$f(s) = f(t) = h(s) = g(t) = f^*(n)$$

其中s是初始节点, t是目标节点, n是s到t的最佳路径上的节点

算法性质:

对有限图, 如果从初始节点s到目标节点t有路径存在, 则算法A一定成功结束

对无限图, 若有从初始节点s到目标节点t的路径, 则A*不结束时, 在OPEN表中即使最小的一个f值也将增到任意大, 或有 $f(n) > f^*(s)$ 。

A*结束前, OPEN表中必存在 $f(n) \leq f^*(s)$ 。

对无限图, 若从初始节点s到目标节点t有路径存在, 则A*一定成功结束。

OPEN表上任一具有 $f(n) < f^*(s)$ 的节点n, 最终都将被A*选作扩展的节点

若存在从初始节点s到目标节点t有路径, 则A*必能找到最佳解结束。

A*选作扩展的任一节点n, 有 $f(n) \leq f^*(s)$ 。

如果 $h_2(n) > h_1(n)$ (目标节点除外), 则A1扩展的节点数 $\geq$ A2扩展的节点数

在定理4中, 评价指标是“扩展的节点数”, 也就是说, 同一个节点无论被扩展多少次, 都只计算一次。

对h的评价方法

- 平均分叉树

设共扩展了d层节点，共搜索了N个节点，则：

其中， b^* 称为平均分叉树。

$$N = \left(1 - b^{*(d+1)}\right) / (1 - b^*)$$

- b^* 越小，说明h效果越好。
- 实验表明， b^* 是一个比较稳定的常数，同一问题基本不随问题规模变化。

改进：

- ☐ 可采纳性不变
- ☐ 不多扩展节点
- ☐ 不增加算法的复杂性

归结原理

子聚集：

无量词约束

元素只是文字的析取

否定符只作用于单个文字

元素间默认为合取

简化子句集

- 只有六种符号： $\forall$, $\exists$; $\rightarrow$, $\vee$, $\wedge$, $\sim$
 - 简化成子句集
 - 1. 消掉蕴含
 - 2. 把否定的都压倒最后去
 - 3. $\forall\exists$, 无关的变量取不同名字, 从而统统移到最左边
 - 4. 去掉存在量词, 无约束变常量, 有约束变函数
 - 5. 把所有的存在量词都去掉
 - 6. 第一层都变成“与”关系, 一个个子句
 - 变成了“与”, 就拆出来
 - 第二层都变成“或”关系
 - 非常像代数式的变换
- $$P \vee (Q \wedge R) \Rightarrow (P \vee Q) \wedge (P \vee R)$$

化子句集的方法

例: $(\exists z)(\forall x)(\exists y)\{[(P(x) \vee Q(x)) \rightarrow R(y)] \vee U(z)\}$

1, 消蕴涵符

理论根据: $a \rightarrow b \Rightarrow \sim a \vee b$

$(\exists z)(\forall x)(\exists y)\{[\sim(P(x) \vee Q(x)) \vee R(y)] \vee U(z)\}$

2, 移动否定符

理论根据: $\sim(a \vee b) \Rightarrow \sim a \wedge \sim b$

$\sim(a \wedge b) \Rightarrow \sim a \vee \sim b$

$\sim(\exists x)P(x) \Rightarrow (\forall x)\sim P(x)$

【南开: 不存在腐败, 任意一个都不腐败】

$\sim(\forall x)P(x) \Rightarrow (\exists x)\sim P(x)$

【天津: 并非所有人都腐败, 存在一个不腐败的】

$(\exists z)(\forall x)(\exists y)\{[(\sim P(x) \wedge \sim Q(x)) \vee R(y)] \vee U(z)\}$

3, 变量标准化

即：对于不同的约束，对应于不同的变量

$$(\exists x)A(x) \vee (\exists x)B(x) \Rightarrow (\exists x)A(x) \vee (\exists y)B(y)$$

4, 量词左移

$$(\exists x)A(x) \vee (\exists y)B(y) \Rightarrow (\exists x) (\exists y) \{A(x) \vee B(y)\}$$

5, 消存在量词 (skolem化)

原则：对于一个受存在量词约束的变量，如果他不受全程量词约束，则该变量用一个常量代替，如果他受全程量词约束，则该变量用一个函数代替。

$$\begin{aligned} & (\exists z) (\forall x)(\exists y) \{[(\sim P(x) \wedge \sim Q(x)) \vee R(y)] \vee U(z)\} \\ \Rightarrow & (\forall x) \{[(\sim P(x) \wedge \sim Q(x)) \vee R(f(x))] \vee U(a)\} \end{aligned}$$

6, 化为合取范式

即 $(a \vee b) \wedge (c \vee d) \wedge (e \vee f)$ 的形式

$$\begin{aligned} & (\forall x) \{[(\sim P(x) \wedge \sim Q(x)) \vee R(f(x))] \vee U(a)\} \\ \Rightarrow & (\forall x) \{(\sim P(x) \wedge \sim Q(x)) \vee R(f(x)) \vee U(a)\} \\ \Rightarrow & (\forall x) \{[\sim P(x) \vee R(f(x)) \vee U(a)] \wedge \\ & [\sim Q(x) \vee R(f(x)) \vee U(a)]\} \end{aligned}$$

7, 隐去全程量词

$$\{[\sim P(x) \vee R(f(x)) \vee U(a)] \wedge [\sim Q(x) \vee R(f(x)) \vee U(a)]\}$$

8, 表示为子句集

$$\{\sim P(x) \vee R(f(x)) \vee U(a), \sim Q(x) \vee R(f(x)) \vee U(a)\}$$

9, 变量标准化 (变量换名)

$$\{\sim P(x_1) \vee R(f(x_1)) \vee U(a), \sim Q(x_2) \vee R(f(x_2)) \vee U(a)\}$$

定理：

若S是合式公式F的子句集，则F永假的充要条件是S不可满足。

S不可满足：若nil属于S，则S不可满足。

4.2 归结方法（命题逻辑）

- 设子句：

$$C_1 = L \vee C_1'$$

$$C_2 = (\sim L) \vee C_2'$$

则归结式C为：

$$C = C_1' \vee C_2'$$

- 定理：

子句集 $S = \{C_1, C_2, \dots, C_n\}$ 与子句集 $S_1 = \{C, C_1, C_2, \dots, C_n\}$ 的不可满足性是等价的。其中，C是 C_1 和 C_2 的归结式。

归结的例子

设公理集：

P,

$(P \wedge Q) \rightarrow R$,

$(S \vee T) \rightarrow Q$,

T

求证：R

- 化子句集：

- $(P \wedge Q) \rightarrow R$

- $\Rightarrow \sim(P \wedge Q) \vee R$

- $\Rightarrow \sim P \vee \sim Q \vee R$

- $(S \vee T) \rightarrow Q$

- $\Rightarrow \sim(S \vee T) \vee Q$

- $\Rightarrow (\sim S \wedge \sim T) \vee Q$

- $\Rightarrow (\sim S \vee Q) \wedge (\sim T \vee Q)$

- $\Rightarrow \{\sim S \vee Q, \sim T \vee Q\}$

子句集：

(1) P

(2) $\sim P \vee \sim Q \vee R$

(3) $\sim S \vee Q$

(4) $\sim T \vee Q$

(5) T

(6) $\sim R$ （目标求反）

子句集:

- (1) P
- (2) $\sim P \vee \sim Q \vee R$
- (3) $\sim S \vee Q$
- (4) $\sim T \vee Q$
- (5) T
- (6) $\sim R$ (目标求反)

归结:

- (7) $\sim P \vee \sim Q$ (2, 6)
- (8) $\sim Q$ (1, 7)
- (9) $\sim T$ (4, 8)
- (10) nil (5, 9)

4.3 谓词逻辑的归结原理

- 问题: 如何找归结对

例: $P(x) \vee Q(y), \sim P(f(y)) \vee R(y)$

$P(A) \vee Q(y), \sim P(f(y)) \vee R(y)$

- 基本概念

– 置换

$s = \{t_1/v_1, t_2/v_2, \dots, t_n/v_n\}$

对公式E实施置换s后得到的公式称为E的例, 记作 E_s 。

例: $s_1 = \{z/x, A/y\}$, 则:

$P[x, f(y), B]_s = P[z, f(A), B]$

- 合一

如果存在一个S置换，使得 $\{E_i\}$ 中

$$E_{1s}=E_{2s}=E_{3s}=\dots=E_{ns},$$

则称 $\{E_i\}$ 是可合一的。S为 $\{E_i\}$ 的合一者。

例： $\{P(x, f(y), B), P(z, f(B), B)\}$

置换 $s=\{A/x, B/y, A/z\}$ 是一个合一者，因为：

$$P(x, f(y), B)_s = P(A, f(B), B)$$

$$P(z, f(B), B)_s = P(A, f(B), B)$$

置换 $s=\{z/x, B/y\}$ 和置换 $s=\{x/z, B/y\}$ 也都是这两个谓词公式的合一者。

结论：合一者不唯一。

- 最一般合一者（mgu）

置换最少，限制最少，产生的例最具一般性。

如前面的例子：

$$\{P(x, f(y), B), P(z, f(B), B)\}$$

对于置换 $\{A/x, B/y, A/z\}$ ，产生的例是：

$$P(A, f(B), B)$$

对于置换 $=\{z/x, B/y\}$ ，产生的例是：

$$P(z, f(B), B)$$

- mgu也不是唯一的。

合一算法

例: $\{P(x, x, z), P(f(y), f(B), y)\}$

前缀表示:

$(P\ x\ x\ z)$

$(P\ (f\ y)\ (f\ B)\ y)$

置换: $\{(f\ y)/x\}$

$(P\ (f\ y)\ (f\ y)\ z)$

$(P\ (f\ y)\ (f\ B)\ y)$

置换: $\{B/y\}$, 并使得 $\{(f\ B)/x\}$

$(P\ (f\ B)\ (f\ B)\ z)$

$(P\ (f\ B)\ (f\ B)\ B)$

置换: $\{B/z\}$

得到置换: $\{(f\ B)/x, B/y, B/z\}$

置换后的结果: $(P\ (f\ B)\ (f\ B)\ B)$

谓词逻辑的归结方法

- 对于子句 $C_1 \vee L_1$ 和 $C_2 \vee L_2$, 如果 L_1 与 $\sim L_2$ 可合一, 且 s 是其合一者, 则 $(C_1 \vee C_2)_s$ 是其归结式。
- 例: $P(x) \vee Q(y), \sim P(f(z)) \vee R(z)$
 $\Rightarrow Q(y) \vee R(z)$

归结举例

设公理集:

$$(\forall x)(R(x) \rightarrow L(x))$$

$$(\forall x)(D(x) \rightarrow \sim L(x))$$

$$(\exists x)(D(x) \wedge I(x))$$

求证: $(\exists x)(I(x) \wedge \sim R(x))$

化子句集:

$$(\forall x)(R(x) \rightarrow L(x))$$

$$\Rightarrow (\forall x)(\sim R(x) \vee L(x))$$

$$\Rightarrow \sim R(x) \vee L(x) \quad (1)$$

$$(\forall x)(D(x) \rightarrow \sim L(x))$$

$$\Rightarrow (\forall x)(\sim D(x) \vee \sim L(x))$$

$$\Rightarrow \sim D(x) \vee \sim L(x) \quad (2)$$

$$(\exists x)(D(x) \wedge I(x))$$

$$\Rightarrow D(A) \wedge I(A)$$

$$\Rightarrow D(A) \quad (3)$$

$$I(A) \quad (4)$$

- 目标求反:

$$\sim(\exists x)(I(x) \wedge \sim R(x))$$

$$\Rightarrow (\forall x) \sim(I(x) \wedge \sim R(x))$$

$$\Rightarrow (\forall x)(\sim I(x) \vee R(x))$$

$$\Rightarrow \sim I(x) \vee R(x) \quad (5)$$

换名后得字句集:

$$\sim R(x_1) \vee L(x_1)$$

$$\sim D(x_2) \vee \sim L(x_2)$$

$$D(A)$$

$$I(A)$$

$$\sim I(x_5) \vee R(x_5)$$

例题的归结树

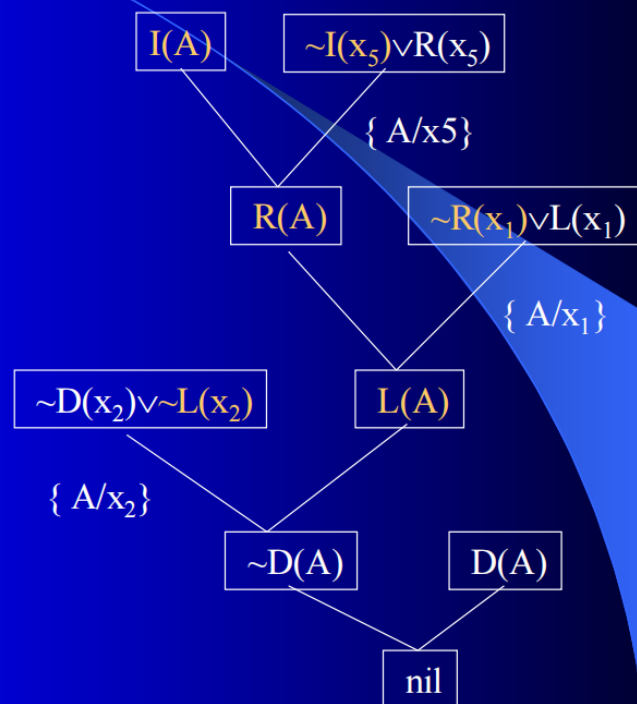
$$\sim R(x_1) \vee L(x_1)$$

$$\sim D(x_2) \vee \sim L(x_2)$$

$$D(A)$$

$$I(A)$$

$$\sim I(x_5) \vee R(x_5)$$



4.4 基于归结的问答系统

- 例：

已知： $(\forall x)[AT(John, x) \rightarrow AT(Fido, x)]$

$AT(John, School)$

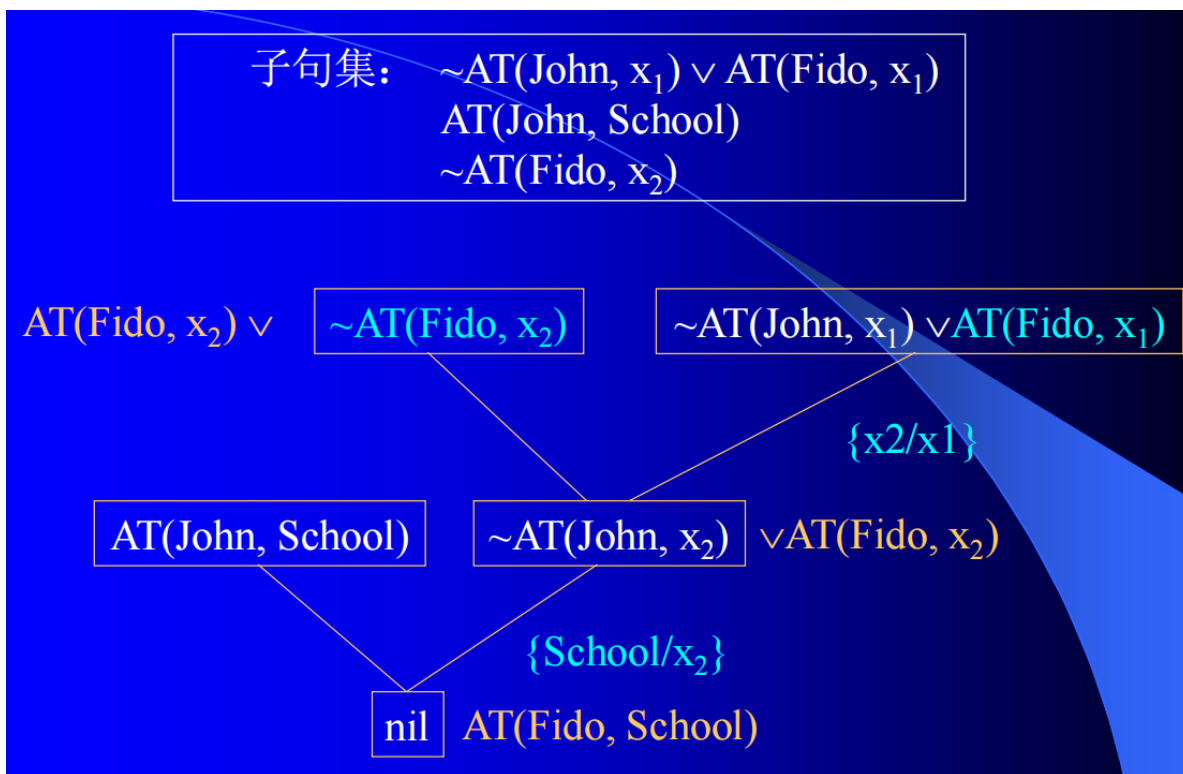
求证： $(\exists x)AT(Fido, x)$

子句集：

$\sim AT(John, x_1) \vee AT(Fido, x_1)$

$AT(John, School)$

$\sim AT(Fido, x_2)$



提取回答的过程

- 先进行归结，证明结论的正确性；
- 用重言式代替结论求反得到的子句；
- 按照证明过程，进行归结；
- 最后，在原来为空的地方，得到的就是提取的回答。
- 修改后的证明树称为修改证明树

- 还有一种办法，就是“或”上一个 ANSWER (x)。寻求NIL变成寻求得到 ANSWER (值)。这时候答案就是 x=值
- 理解成，只有 x=值 的时候，才推出矛盾因为用的是反证法，这时才成立

已知：

1, $\sim \text{ON}(s_0)$

2, $(\forall x)(\forall s)(\sim \text{ON}(s) \rightarrow \text{AT}(\text{box}, x, \text{push}(x, s)))$

3, $(\forall s)(\text{ON}(\text{climb}(s)))$

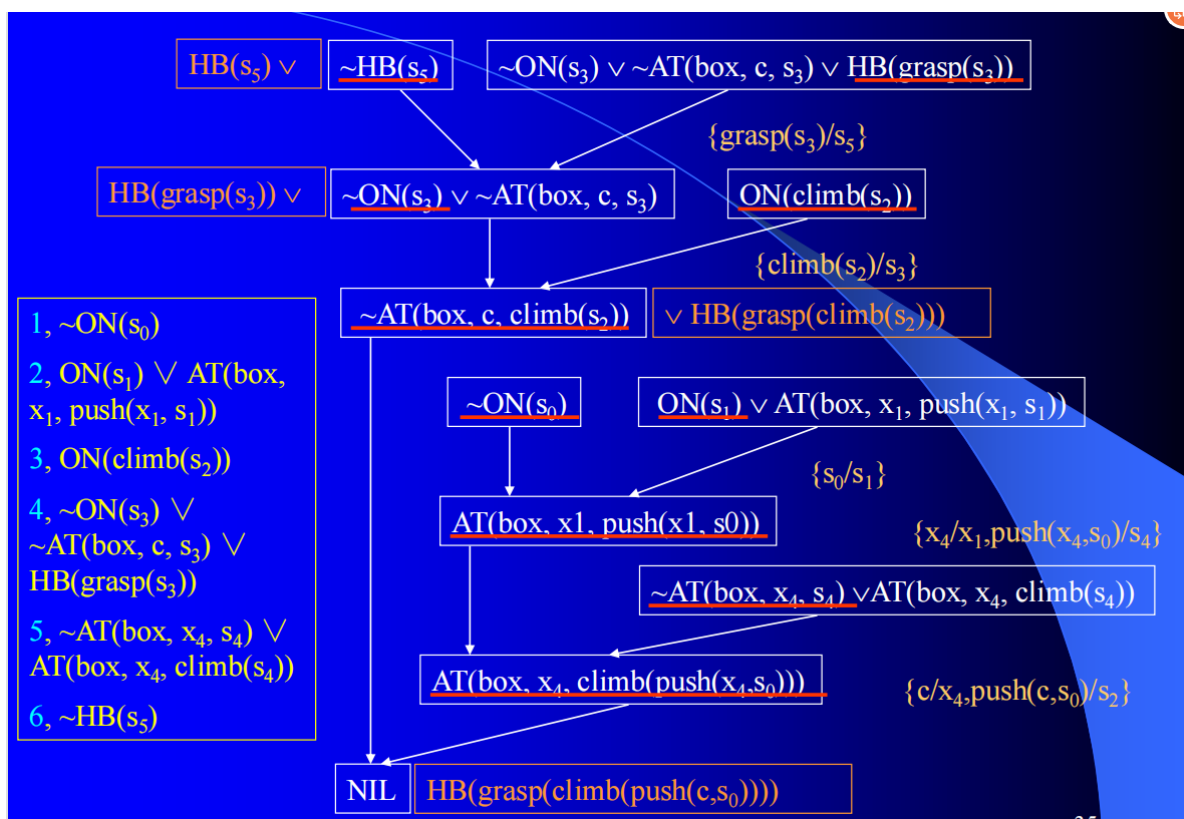
4, $(\forall s)((\text{ON}(s) \wedge \text{AT}(\text{box}, c, s)) \rightarrow \text{HB}(\text{grasp}(s)))$

5, $(\forall x)(\forall s)(\text{AT}(\text{box}, x, s) \rightarrow \text{AT}(\text{box}, x, \text{climb}(s)))$

求解： $(\exists s)\text{HB}(s)$

问题的子句集

- 1, $\sim \text{ON}(s_0)$
- 2, $\text{ON}(s_1) \vee \text{AT}(\text{box}, x_1, \text{push}(x_1, s_1))$
- 3, $\text{ON}(\text{climb}(s_2))$
- 4, $\sim \text{ON}(s_3) \vee \sim \text{AT}(\text{box}, c, s_3) \vee \text{HB}(\text{grasp}(s_3))$
- 5, $\sim \text{AT}(\text{box}, x_4, s_4) \vee \text{AT}(\text{box}, x_4, \text{climb}(s_4))$
- 6, $\sim \text{HB}(s_5)$



博弈论：

均衡收益是低效的

优策略：

对于玩家 I 来说，如果不管其他玩家怎么做，玩 x 都比玩 y 好，那么策略 x 支配策略 y

你永远不应该玩严格控制的策略

优势策略均衡

双方都选择对自己最优的 是低效的

理性玩家的理性游戏会导致不好的结果。

- 如果没有主导策略呢？
 - 没有任何一种策略可以支配其他所有策略
 - 但是一些策略可能仍然占主导地位
- 假设每个人都知道每个人是理性的。
 - 可以移除他们主导的策略
 - 可能会揭示一种新的主导策略

第三课:如果你没有一个主导的策略，设身处地为对手着想，试着预测他们会怎么做。例如，站在他们的立场上，他们不会选择被支配的策略。 **

第四课 要弄清楚在游戏中你应该选择什么动作，一个好的第一步是弄清楚你的收益是什么（你关心什么）以及其他玩家的收益是什么。

纳什均衡（nash）

一套策略，每个玩家一套，这样在其他玩家不偏离的情况下，没有玩家有动力偏离她的策略

主导战略均衡是纳什均衡。

混合策略纳什均衡

每个有限对策都有一个混合策略纳什均衡

零和和非零和

零和游戏

- 代理具有相反的效用。
- 纯粹的竞争:有赢有输, 其 sumis “零”。



非零和游戏

- 代理有独立的公用事业。
- 合作, 冷漠, 竞争, ...
- 双赢, 赢输或输输, 其总和不是“零”。



搜索和游戏

- 搜索——没有对手
 - 解决方法是寻找目标的(启发式)方法
 - 启发式和 CSP 技术可以找到最佳的解决方案
 - 评估功能:从开始到目标到给定节点的成本估算
 - 示例:路径规划、活动调度
- 游戏-对手
 - 解决方案就是策略
 - 策略为每一个可能的对手回复指定移动。
 - 时间限制迫使近似解
 - 评价功能:评价游戏位置的“优劣”
 - 例子:国际象棋, 跳棋, 奥赛罗, 西洋双陆棋

极大极小搜索:

极小极大算法常用于二人博弈游戏, 目的是寻找最优的方案使得自己能够利益最大化。基本思想就是假设自己 (A) 足够聪明, 总是能选择最有利于自己的方案, 而对手 (B) 同样足够聪明, 总会选择最不利 A 的方案。

该算法使用深度优先搜索 (Depth First Search) 遍历决策树来填充树中间节点的利益值, 叶子节点的利益值通常是通过一个利益评估函数算得。

剪枝算法

假设 α 为下界， β 为上界，对于 $\alpha \leq N \leq \beta$:

若 $\alpha \leq \beta$ 则N有解。

若 $\alpha > \beta$ 则N无解。

1) 任何max节点的alpha值大于其父节点的beta值，则把该节点剩余的枝剪去，因为这说明在该节点之前至少有一个节点的alpha值比它的上界要小，而父节点是min节点，所以这个节点明显不是min节点所想要的。或者说这个节点对对手有利。

2) 同理任何min节点的beta值小于其父节点的alpha值，则把该节点剩余的枝剪去。

蒙特卡罗树搜索：

1. **选择** (Selection)
2. **扩展** (expansion)
3. **模拟** (Simulation)
4. **回溯** (Backpropagation)

机器学习基础：

● 监督学习

监督学习是指：利用一组已知类别的样本调整 分类器 的 参数 ，使其达到所要求性能的过程，也称为 监督训练或有教师学习。． 监督学习是从标记的训练数据来推断一个功能的机器学习任务。． 训练数据包括一套训练示例。． 在监督学习中，每个实例都是由一个输入对象（通常为矢量）和一个期望的输出值（也称为监督信号）组成。． 监督学习算法是分析该训练数据，并产生一个推断的功能，其可以用于映射出新的实例。． 一个最佳的方案将允许该算法来正确地决定那些看不见的实例的类标签。． 这就要求学习算法是在一种"合理"的方式从一种从训练数据到看不见的情况下形成

● 无监督学习

监督学习是一种目的明确的训练方式，你知道得到的是什么；而无监督学习则是没有明确目的的训练方式，你无法提前知道结果是什么。

监督学习需要给数据打标签；而无监督学习不需要给数据打标签。

监督学习由于目标明确，所以可以衡量效果；而无监督学习几乎无法量化效果如何。

监督学习 VS 无监督学习

监督学习

1. 目标明确
2. 需要带标签的训练数据
3. 效果容易评估

无监督学习

1. 目标不明确
2. 不需要带标签的数据
3. 效果很难评估

● SVM

支持向量机（Support Vector Machine, SVM）是一类按监督学习（supervised learning）方式对数据进行二元分类的广义线性分类器（generalized linear classifier），其决策边界是对学习样本求解的最大边距超平面（maximum-margin hyperplane） [1-3] 。

SVM使用铰链损失函数（hinge loss）计算经验风险（empirical risk）并在求解系统中加入了正则化项以优化结构风险（structural risk），是一个具有稀疏性和稳健性的分类器 [2] 。SVM可以通过核方法（kernel method）进行非线性分类，是常见的核学习（kernel learning）方法之一 [4] 。

SVM被提出于1964年，在二十世纪90年代后得到快速发展并衍生出一系列改进和扩展算法，在人像识别、文本分类等模式识别（pattern recognition）问题中有得到应用

● PCA

PCA（Principal Component Analysis） 是一种常见的数据分析方式，常用于高维数据的降维，可用于提取数据的主要特征分量。

PCA 的数学推导可以从最大可分型和最近重构性两方面进行，前者的优化条件为划分后方差最大，后者的优化条件为点到划分平面距离最小，

● K-Means

K-means 是我们最常用的基于欧式距离的聚类算法，其认为两个目标的距离越近，相似度越大。

K-means 有一个著名的解释：牧师-村民模型： 我们可以看到该牧师的目的是为了每个村民到其最近中心点的距离和最小。

● LDA

LDA(线性判别算法)

不同于PCA方差最大化理论，LDA算法的思想是将数据投影到低维空间之后，使得同一类数据尽可能的紧凑，不同类的数据尽可能分散。因此，LDA算法是一种有监督的机器学习算法。同时，LDA有如下两个假设：

- (1) 原始数据根据样本均值进行分类。
- (2) 不同类的数据拥有相同的协方差矩阵。

当然，在实际情况中，不可能满足以上两个假设。但是当数据主要是由均值来区分的时候，LDA一般都可以取得很好的效果。

感知器

● 基本感知器

感知器是人工神经网络中的一种典型结构， 它的主要的特点是结构简单，对所能解决的问题 存在着收敛算法，并能从数学上严格证明，从而对神经网络研究起了重要的推动作用。

● 非线性的激活函数

非线性激活函数可以使神经网络随意逼近复杂函数，类似非线性的**Sine**函数随意逼近各种复杂函数一样。 没有激活函数带来的非线性，多层神经网络和单层无异。 非线性激活函数对深度神经网络的函数逼近能力起着至关重要的作用

深度神经网络

- 感知器堆叠
- 损失函数
- 反向传播

模型训练

- 自适应学习率
- Batching
- 过拟合

基础知识

- 机器视觉
- 卷积提取特征

CNN

- 基本网络结构
- 多层卷积、池化
- 分类问题

多种应用

- 目标检测
- 图像分割
- 医学、机器人

RNN

- 适用于序列模型的任务
- BPTT
- 梯度爆炸和梯度消失

LSTM

- 门控单元

- 长期状态的记忆

应用

- 文本生成
- 音乐生成
- 机器翻译
- 情感分类

自编码器

- 监督学习与无监督学习
- 编码与解码

VAEs

- 学习输入的概率分布
- 正态分布的潜在空间和隐变量
- 生成输入的重构

GAN

- 生成器和判别器竞争
- 各种变形强大的生成能力
- PGGAN
- styleGAN

强化学习

- 交互中学习
- Q-function
- Q-Learning

DQN

- 深度神经网络拟合Q函数
- Value-base

策略梯度

- 直接优化policy函数
- 连续的动作空间映射

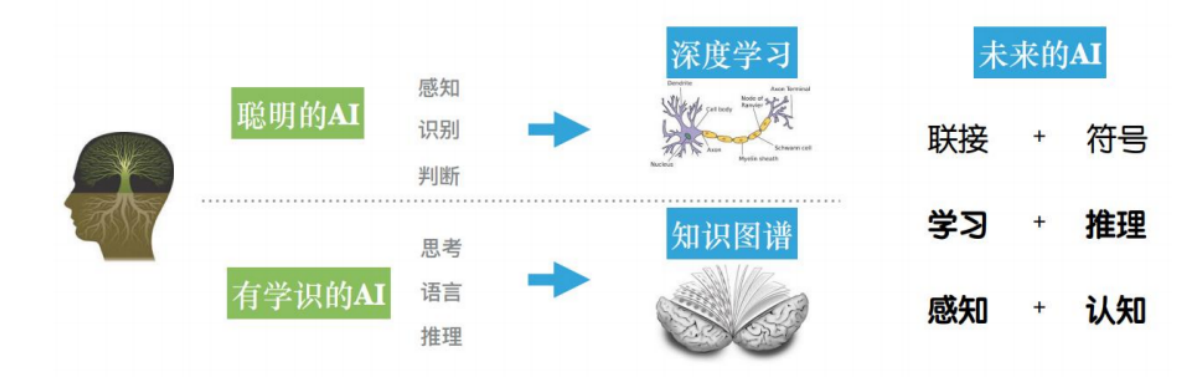
知识图谱

什么是知识图谱

2009年，Markram在TED大会上，向公众宣布了一个惊人大计划：他要在了解大脑结构的基础上，用计算机创建一个复杂的数学模型，模拟人脑的86亿个神经元和100万亿的突触。

Markram表示，研究一旦成功将对整个人类带来极大意义：为老年痴呆等疾病带来革命性进展，研发出更智能认知能力更强的机器人，甚至帮助找到让计算机处理速度更快的方式。

欧盟政府牵头，联合多个国家和地区的科研机构，打算资助这个看起来极其炫酷的实验，他们重金押注，一出手就是13亿欧元，将蓝脑计划更名为“人类脑计划”，全称Human Brain Project，简称为HBP



- > 知识图谱由Google于2012年提出
- > 知识工程，专家系统

知识图谱

知识图谱（Knowledge graph）是一种以结构化的组织形式描述客观世界中概念、实体及其关系，将数据表达成更接近人类认知世界的形式。

知识图谱三元组

知识图谱，是一种揭示实体之间关系的语义网络。知识图谱是由一条条知识组成，每条知识表示为一个SPO三元组

(Subject-Predicate-Object)

知识图谱构建方式

知识图谱的构建方式有两种，一种是**自顶向下(top-down)**的构建方式。自顶向下指的是先为知识图谱定义好本体与数据模式，再将实体加入到知识库。另外一种**是自底向上(bottom-up)**的构建方式。自底向上指的是从一些开放链接数据中提取出实体，选择其中置信度较高的加入到知识库，再构建实体与实体之间的联系。

知识图谱架构

知识图谱在逻辑结构上可分为模式层与数据层两个层次，数据层主要是由一系列的事实组成，而知识将以事实为单位进行存储。模式层构建在数据层之上，主要是通过本体库来规范数据层的一系列事实表达。本体是结构化知识库的概念模板，通过本体库而形成的知识库不仅层次结构较强，并且冗余程度较小

知识图谱的表示

基于离散符号的知识表示

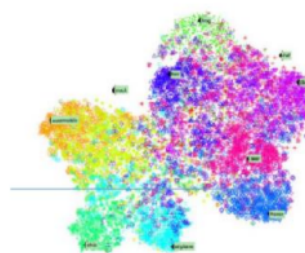
RDF, OWL, 各种Rule Language等



显式知识、强逻辑约束、易于解释、推理不易扩展

基于连续向量的知识表示

Tensor, 各种Embedding, 神经网络表示等



隐式知识、弱逻辑约束、不易解释、对接神经网络

VS

- 语义网
 - XML
 - RDF
 - OWL

XML最初被设计用于网络之间数据的传输，语法类似于html，但是可以自行定义标签的名字。这个特点非常适合定义符合各自要求的数据格式，也使得xml具有更强大的表达能力，不过因此也导致XML数据的结构过于松散随意，其统一性和通用性受到了严重限制。读者通常需要对xml Schema文件（xsls）有足够的详细了解之后才可以完全理解xml文件背后的语义信息。

- 语义网
 - XML
 - RDF
 - OWL

RDF（Resource Description Framework，资源描述框架）是一种资源描述语言，它受到元数据标准、框架系统、面向对象语言等多方面的影响，被用来描述各种网络资源，其出现为人们在Web上发布结构化数据提供一个标准的数据描述框架。RDF提出了一个简单的二元关系模型来表示事物之间的语义关系，即使用三元组集合的方式来描述事物和关系，三元组的结构为“资源-属性-属性值”，资源实体由URI表示，属性值可以是另一个资源实体的URI，也可以是某种数据类型的值，也称为literals（字面量）。

- 语义网
 - XML
 - RDF
 - **OWL**

OWL（网络本体语言），编程语言，全称Web Ontology Language，是W3C开发的一种网络本体语言，用于对本体进行语义描述。OWL被设计用来处理资讯的内容而不是仅仅向人类呈现信息的应用。通过提供更多具有形式语义的词汇，使之在Web内容的机器可理解性方面要强于XML、RDF和RDF Schema（RDF-S）。

数值化表示

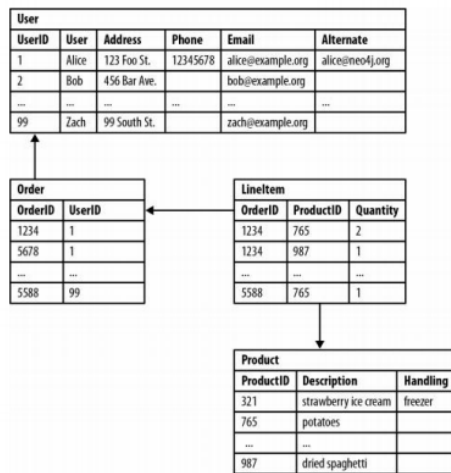
表示学习旨在将研究对象的语义信息表示为稠密低维实值向量，知识表示学习主要是面向知识图谱中的实体和关系进行表示学习。使用建模方法将实体和关系表示在低维稠密向量空间中，然后进行计算和推理。简单来说，就是将三元组表示成向量的这个过程就称为表示学习。

1. DeepWalk
2. Node2vec
3. Struc2vec
4. LINE
5. SDNE
6. transE
7. transH

知识图谱的存储与查询

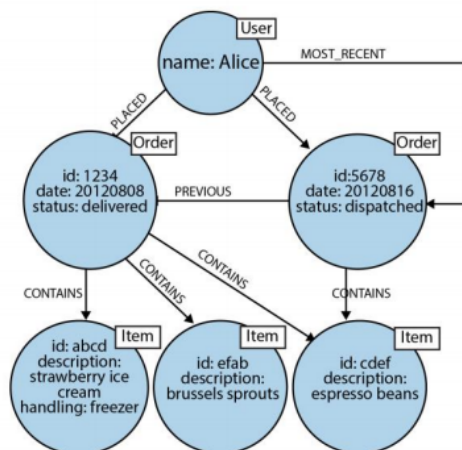
名称	存储格式
WikiData	JSon、Rdf、ttl
Freebase	RDF
ConceptNet	JSON- LD
DBPedia	RDF

关系数据库局限性



- 关系模型将语义关联关系隐藏在外键结构中，无显示表达，并带来关联查询与计算的复杂性。
- 数据来源多样性带来大量**离群数据 (Outlier Data)**，导致数据集的宏观结构愈发复杂和不规整，对于包含大量离群数据的场景，关系模型将造成大量表连接、稀疏行和非空处理。
- 互联网的开放世界假设要求数据模型满足**高动态和去中心化的扩增数据**的能力，关系模型对表结构的范式要求限制了Schema层的动态性。

图数据库



- 自然表达：图是十分自然的描述事物关系的方式，更加接近于人脑对客观事物的记忆方式。
- 易于扩展：图模型更加易于适应变化，例如在图中，临时希望获取历史订单，只需新增边即可。
- 复杂关联表达：图模型易于表达复杂关联逻辑的查询，例如在推荐系统中，希望表达复杂的推荐逻辑，例如：“all the flavors of ice cream liked by people who enjoy espresso but dislike Brussels sprouts, and who live in a particular neighborhood.”
- 多跳优化：在处理多跳查询上，图模型有性能优势。

知识抽取



应用

知识推理

- 基于描述逻辑的推理（如DL-based）
- 基于图结构和统计规则挖掘的推理（如：PRA、AMIE）
- 基于知识图谱表示学习的推理（如：TransE、GCN）
- 基于概率逻辑的方法（如：Statistical Relational Learning）

典型知识图谱项目

- Cyc是在1984年由Douglas Lenat开始创建。最初的目标是要建立人类最大的常识知识库。典型的常识知识如“Every tree is a plant”，“Plants die eventually”等。
- Cyc知识库主要由术语Terms和断言Assertions组成。Terms包含概念、关系和实体的定义。Assertions用来建立Terms之间的关系，这既包括事实Fact描述，也包含规则Rule的描述。
- 最新的Cyc知识库包含有50万条Terms和700万条Assertions。Cyc的主要特点是基于形式化的知识表示方法来刻画知识。形式化的优势是可以支持复杂的推理。但过于形式化也导致知识库的扩展性和应用的灵活性不够。Cyc提供开放版本OpenCyc。

WordNet

- WordNet是最著名的词典知识库，主要用于词义消歧。WordNet 由普林斯顿大学认识科学实验室从1985年开始开发。
- WordNet主要定义了名词、动词、形容词和副词之间的语义关系。例如名词之间的上下位关系（如：“猫科动物”是“猫”的上位词），动词之间的蕴含关系（如：“打鼾”蕴含着“睡眠”）等。
- WordNet3.0已经包含超过15万个词和20万个语义关系。

ConceptNet

- ConceptNet是常识知识库。最早源于MIT媒体实验室的Open Mind Common Sense (OMCS)项目。OMCS项目是由著名人工智能专家Marvin Minsky于1999年建议创立。
- ConceptNet主要依靠互联网众包、专家创建和游戏三种方法来构建。新版本导入大量开放的结构化数据，如DBPedia、Wikinary、Wordnet等
- ConceptNet知识库以三元组形式的关系型知识构成。ConceptNet5版本已经包含有2800万关系描述。与Cyc相比，ConceptNet采用了非形式化、更加接近自然语言的描述，而不是像Cyc那样采用形式化的谓词逻辑。
- 与链接数据和谷歌知识图谱相比，ConceptNet比较侧重于词与词之间的关系。从这个角度看，ConceptNet更加接近于WordNet，但是又比WordNet包含的关系类型多。此外，ConceptNet完全免费开放，并支持多种语言。





2021/12/23

WikiData

人工智能:

研究如何用计算机来表示和执行人类的智能活动，以模拟人脑所从事的推理、学习、思考和规划等思维活动，并解决需要人类的智力才能处理的复杂问题等。

定义

n 智能机器：能够在各类环境中自主地或交互地执行各种拟人任务的机器。

n 人工智能（学科）：人工智能(学科)是计算机科学中涉及研究、设计和应用智能机器的一个分支。它的近期主要目标在于研究用机器来模仿和执行人脑的某些智力功能，并开发相关理论和技术。

n 人工智能（能力）：人工智能(能力)是智能机器所执行的通常与人类智能有关的智能行为，如判断、推理、证明、识别、感知、理解、通信、设计、思考、规划、学习和问题求解等思维活动。

【注】

n 固定了算法的，基本上不算AI。（目前世界上90%的计算都不属于AI范畴）

n 自动控制类（飞机自动驾驶），科学计算类（石油勘探，天气预报，办公室自动化等）。

历史

n 1950年，由艾伦图灵提出了计算机与智能的概念（图灵测试，图灵机）

n 1956年，达特茅斯会议，正式提出人工智能（Artificial Intelligence）的概念

分类

n 弱人工智能，擅长于单个方面的人工智能。

n 强人工智能，类似人类级别的人工智能，在各个方面都能和人类比肩的人工智能

n 超人工智能，在几乎所有领域都比最聪明的人类大脑都聪明很多的人工智能。

I 三大流派：

n 符号主义：把现实的物，映射到代表它的符号，在符号上完成所有的推理，计算等

n 连接主义：人脑就是神经元之间的连接（神经网络模型）

n 行为主义：只能来自更低级的感知和行动，表现的好就行

I 发展阶段

n 20世纪50年代末-70年代初

u 人工智能诞生

n 20世纪80年代初-90年代初

u 数学模型重大突破，诞生了很多神经网络、反向传播算法

n 21世纪至今

u 大数据、算法模型重大突破，深度神经网络提出

n 新一代人工智能与前两轮浪潮的本质区别：

u 新一代人工智能技术已经可以大规模应用于经济社会时间，给企业带来盈利，给社会管理带来实质性改善

u 从学术驱动、研究驱动转变为需求驱动和产业驱动

n 传统人工智能：自动化、电子、计算机之间的

n 新人工智能：计算机与软件之间的

I 相关领域

n 机器定理证明

n 博弈

n 模式识别

n 自然语言处理

n 数据挖掘和知识发现

n 专家系统

图灵机：

I 七元组：

n 纸带，符号，读写头，规则，状态，起始，结束（存储，符号，读写，程序，数据，开始，结束）

I 停机问题：

n 判断任意一个[程序](#)是否能在有限的时间之内结束运行的问题。该问题等价于如下的判定问题：是否存在一个程序P，对于任意输入的程序w，能够判断w会在有限时间内结束或者死循环。

n 可计算性：并不是每一个函数都可以被描述为一个算法

n 哥德尔定理：

u 任何强壮的数学系统必定包含一个真的陈述句，然而在该系统中，这个陈述句却是不可证明的。（任何一个公理系统，都存在一些命题无法被证明是真还是假）

u 第一定理：任意一个包含一阶谓词逻辑与初等数论的**形式系统**，都存在一个命题，它在这个系统中既不能被证明为真，也不能被证明为否。

u 第二定理：如果系统S含有初等数论，当S无矛盾时，它的无矛盾性不可能在S内证明。

n 康托尔对角线论证法：用于说明实数集合是不可数集的证明。

l 图灵测试：

n 屋子里有人、计算机，外边的人提问，交互用打印机、电话等方式，如果提问者无法分辨是人回答的还是计算机回答的，就是通过了图灵测试

l 摩尔定理

n 当价格不变时，集成电路上可容纳的元器件的数目，约每个18-24个月便会增加一倍，性能也将提升一倍

l

大数据：

l 指无法在一定时间范围内用常规软件工具进行捕捉、管理和处理的数据集合，是需要新处理模式才能具有更强的决策力、洞察发现力和流程优化能力的海量、高增长率和多样化的信息资产。（巨大量的数据）（相关性，非因果性）

l 特点

n 大量 (volume)

n 高速 (velocity)

n 多样 (variety)

n 低价值密度 (value)

n 真实性 (veracity)

n 【注】：大数据时代大量使用相关性而不是因果性

l 分类

n 局部大数据：部分领域内的数据

n 打通的大数据：

专家系统（产生式系统）：

l 组成

n 综合数据库（信息）

n 产生式规则（知识）

n 控制系统（规则的解释或执行程序）

l 特点：

n 数据驱动

n 知识无序

n 控制系统与问题无关

n 数据、知识和控制相互独立

l 类型

n 正向、逆向、双向产生式系统

n 可交换的产生式系统

n 可分解的产生式系统

l 搜索策略：

n 内容

u 状态空间的搜索问题

n 方式

u 盲目式搜索

u 启发式搜索

n 关键

u 利用知识，尽可能有效的找到问题的解

n 回溯（递归）策略

u 深度问题（深度加以限制）

u 死循环问题（记录通过的路径状态）

n 图搜索：保留所有已搜索过的路径

u 基本概念

l 节点深度

n 根节点，0

n 其他节点：父节点+1

l 路径

n 节点序列

n 散耗值：路径中所有节点散耗值的总和

n 扩展结点：生成该节点的所有后继节点，并给出他们的散耗值

u 无信息图搜索过程

l 深度优先：

n 一般不能保证找到最优解；

n 当深度限制不合理时，可能找不到解；

n 最坏情况时，搜索空间等于穷举；

n 与回溯法的差别：图搜索

n 通用的与问题无关的方法

l 宽度优先：

n 当问题有解时，一定能够找到解；

n 当问题为单位散耗值，且问题有解时，一定能找到最优解；

n 方法与问题无关，具有通用性；

n 效率较低；

n 属于图搜索方法

l 渐进式深度优先搜索方法：

n 目的

u 解决宽度优先方法的空间问题和回溯方法不能找到最优解的问题

n 思想

u 给回溯法一个较小的深度限制，逐步渐增增加深度限制，直到找到解或找遍所有分支。（解决宽度优先方法的空间问题和回溯方法不能找到最优解的问题）

l 启发式图搜索

n 利用知识引导搜索，减少搜索范围，降低问题复杂度。

n 启发信息的强度

u 启发强度强：降低搜索工作量，但可能导致找不到最优解

u 启发强度弱：工作量极大，极限情况下盲目搜索，但可能找到最优解

n 思想

u 定义评价函数 f ，对当前的搜索状态进行评估，找到一个最有希望的结点来拓展

u f （启发函数）= $g + h$ （评价函数）

n 算法：

u A*算法

l $f(n) = g(n) + h(n)$ ，且 $h(n) \leq h^*(n)$

l $f(s) = f(t) = h(s) = g(t) = f^*(n)$

l 定理

n 对有限图，如果从初始节点 s 到目标节点 t 有路径存在，则算法A一定成功结束

u 对无限图，若有从初始节点 s 到目标节点 t 的路径，则A不结束时，在OPEN表中即使最小的一个 f 值也将增到任意大，或有 $f(n) > f(s)$ 。

u A结束前，OPEN表中必存在 $f(n) \leq f(s)$ 。

n 对无限图，若从初始节点 s 到目标节点 t 有路径存在，则A*一定成功结束

u OPEN表上任一具有 $f(n) < f(s)$ 的节点 n ，最终都将被A选作扩展的节点。

n 若存在从初始节点 s 到目标节点 t 有路径，则A*必能找到最佳解结束。

u A选作扩展的任一节点 n , 有 $f(n) \leq f(s)$ 。

n 如果 $h_2(n) > h_1(n)$ (目标节点除外), 则 A_1 扩展的节点数 $\geq A_2$ 扩展的节点数

n 队友线图

重点

归结原理

l 定理证明方法, 从理论上解决了定理证明的问题

l 归结原理的题出对机器定理证明问题起到了推动作用

l 若 S 是合取公式 F 的子句集, 则 F 永假的充要条件是 S 不可满足 ($\text{nil} \in S$) (一种定理证明方法, 从理论上解决了定理证明问题)

l 使用归结原理证明定理的思路

n 目标的否定连同已知条件一起, 化为子句集, 并给出一种变换方法, 使得 $S \rightarrow S_1 \rightarrow S_2 \rightarrow \dots \rightarrow S_n$ 同时保证当 S_n 不可满足时, 有 S 不可满足。

l 子句集

n 无量词约束

n 否定符只作用于单个文字

n 元素默认为合区

n 化子句集的方法

u 消除蕴含符

l $a \Rightarrow b = \sim a \vee b$

u 移动否定符号

l $\sim(a \vee b) \Rightarrow \sim a \wedge \sim b$

l $\sim(a \wedge b) \Rightarrow \sim a \vee \sim b$

l $\sim(\exists x)P(x) \Rightarrow (\forall x)\sim P(x)$

u 变量标准化

l 对不同的约束, 对应于不同的变量

l $(\exists x)A(x) \vee (\exists x)B(x) \Rightarrow (\exists x)A(x) \vee (\exists y)B(y)$

u 量词左移

l $(\exists x)A(x) \vee (\exists y)B(y) \Rightarrow (\exists x)(\exists y)\{A(x) \vee B(y)\}$

u 消除存在量词

l 对于一个受存在量词约束的变量, 如果他不受全程量词约束, 则该变量用一个常量代替, 如果他受全程量词约束, 则该变量用一个函数代替。

l $(\exists z)(\forall x)(\exists y)\{[(\sim P(x) \wedge \sim Q(x)) \vee R(y)] \vee U(z)\} \Rightarrow (\forall x)\{[(\sim P(x) \wedge \sim Q(x)) \vee R(f(x))] \vee U(a)\}$

u 化为合取范式

u 隐去全称量词

u 表示为子句集

u 变量标准化（变量换名）

l 谓词逻辑的归结原理

n 关键：找到归结对

n 基本概念

u 置换

l $s = \{t_1/v_1, t_2/v_2, \dots, t_n/v_n\}$ 对公式 E 实施置换 s 后得到的公式称为 E 的例，记作 E_s 。

l $s_1 = \{z/x, A/y\}$ ，则： $P[x, f(y), B]_s = P[z, f(A), B]$

u 合一

l 如果存在一个 S 置换，使得 $\{E_i\}$ 中 $E_1s = E_2s = E_3s = \dots = E_ns$ ，则称 $\{E_i\}$ 是可合一的。 S 为 $\{E_i\}$ 的合一者。

l $\{P(x, f(y), B), P(z, f(B), B)\}$ 置换 $s = \{A/x, B/y, A/z\}$ 是一个合一者，因为： $P(x, f(y), B)_s = P(A, f(B), B)$ $P(z, f(B), B)_s = P(A, f(B), B)$

l 【注】：合一者不唯一

l 最一般合一者 (mgu)：置换最少，限制最少，产生的例子最具一般性

l 合一算法

n 谓词逻辑的归结方法

u 对于子句 $C_1 \vee L_1$ 和 $C_2 \vee L_2$ ，如果 L_1 与 $\sim L_2$ 可合一，且 s 是其合一者，则 $(C_1 \vee C_2)_s$ 是其归结式。

n 提取回答的过程

u 进行归结，证明结论的正确性

u 用重言式代替结论求反得到子句

u 按照证明过程，进行归结

u 在原来为空的地方得到的就是提取的回答

u 修改后的证明树被称为修改证明树

机器学习：寻找一个映射函数（最优的）

分类：监督学习（给定目标），非监督学习

深度学习：深度 $\rightarrow$ 许多隐藏层

步骤：定义函数集，评价函数，选择最优的函数

神经网络结构：全连接神经网络，卷积神经网络，循环神经网络

博弈论

I 定义

n 关于制定决策，具体而言，理性决策者之间冲突和合作的数学模型

I 应用

n 经济，政治，物理，逻辑学，计算机科学，生物学

n 行为关系，决策科学

I 特点

n 至少两个玩家

n 轮流 / 同步

n 信息完全可知 / 信息不完全可知

n 确定的 / 随机的

n 合作的 / 竞争的

n 零和 / 非零和

u 零和

I 目标对立

I 纯竞争关系

I 结果的总和为0

u 非零和

I 又相互独立的目标

I 关系可以为合作，竞争，冷漠

I 结果总和不为0

I 博弈树：

n 确定的，信息完全可知的，轮流的，两个玩家，零和

n 两个玩家分别为：Max，Min

n 结点为局面，边为动作

I 搜索与博弈

n 搜索

u 没有对手

u 解决方案是采用启发式方法去寻找目标

u 启发式和CSP技术能够找到最佳方案

u 评估函数：从开始到结束的总耗费

u 路径规划，活动调度

n 博弈

u 解决方案是策略（特指每一步得到的对手的反馈）

u 因为时间要求，强制得到一个近似解

u 评估函数：评估决策的好坏

u 下棋

l 极小极大搜索

l $\alpha\beta$ 剪枝：

n 将min层选择的值作为 β ，将max层选择的值作为 α

n min层向下穿 β ，max层向下穿 α

n α 剪枝：min结点的 β 值小于等于父节点的 α 值

n β 剪枝：max结点的 α 值大于等于父节点的 β 值

l 蒙特卡罗树搜索

n 选择：从根节点 R 开始，递归选择最优的子节点（后面会解释）直到达到叶子节点 L。

$$v_i + C \times \sqrt{\frac{\ln N}{n_i}}$$

n 扩展：如果 L 不是一个终止节点（也就是，不会导致博弈游戏终止）那么就创建一个或者更多的子节点，选择其中一个 C。

n 仿真：从 C 开始运行一个模拟的输出，直到博弈游戏结束。

n 反向传播：用模拟的结果输出更新当前行动序列

机器学习

l 思想：学习是一个系统通过经验提升能力的过程

l 机器学习是一个关于<P, T, E>的算法（P, 表现；T, 任务；E, 经验）

l 传统程序：数据+程序→输出

l 机器学习：数据+输出→程序（寻找一个函数）

l 步骤

n 获得一个函数集

n 训练数据

n 挑选一个优函数

l 学习也就是泛化

l 学习的种类

n 监督学习

u 训练数据+期望输出

u 回归

u 分类

- n 非监督学习

- u 训练数据

- u 聚类

- n 半监督学习

- u 训练数据+很少的期望输出

- n 强化学习

- u 对行为有反馈

- u 玩游戏

- u 平衡球

- u 信用预估

- l 搭建学习系统

- n 选择训练领域

- n 选择学习目标 (靶向函数)

- n 选择如何表示目标

- n 选择一个算法

深度学习

- l 步骤

- n 定义函数集

- n 评估函数

- n 挑选最有的函数